17

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

의존성 주입과 제어의 역전

제2부 NestJS 핵심 · 5장 의존성 주입

서비스가 쓸 객체를 직접 만들지 않으면 달라지는 것, Nest가 그 객체를 고르는 방법

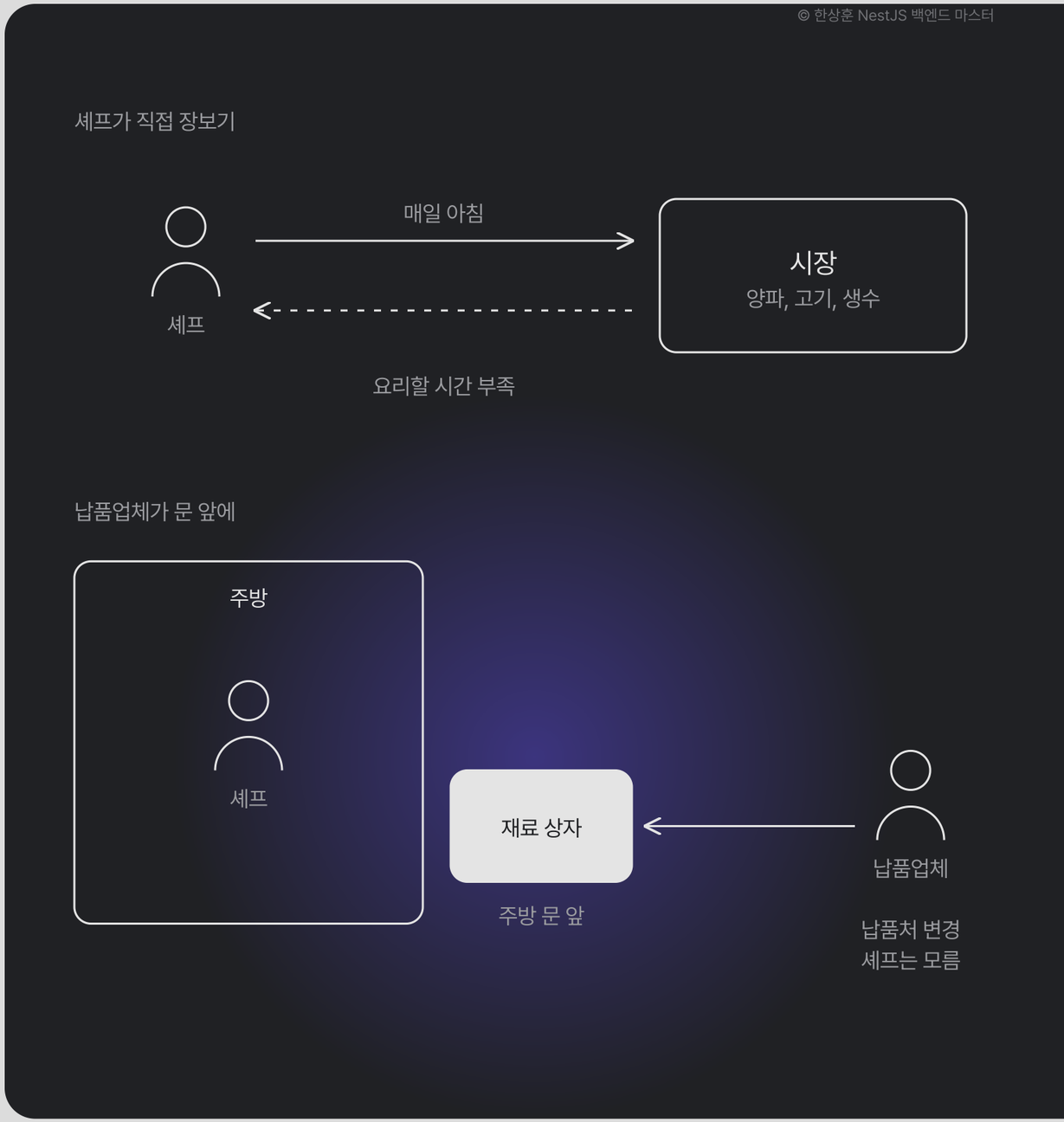


셰프와 납품업체

→ 셰프가 매일 아침 시장에서 장보기: 요리할 시간 부족

→ 납품업체: 정해진 재료를 주방 문 앞에

→ 양파 납품처 변경: 셰프는 모름, 문 앞의 양파만 확인



© 한상훈 NestJS 백엔드 마스터

서비스는 Nest가 하나 만들어서

→ 컨트롤러마다 `new SeatsService()`: 인스턴스 둘, 창구에서 좌석 없음

→ `providers` 등록 + 생성자: 인스턴스 하나

"규칙은 서비스에 두고, 서비스는 Nest가 하나 만들어서 필요한 곳에 넣어 줍니다."

→ 남은 질문: Nest가 무엇을 보고 넣는지, 받아 쓰면 무엇이 나은지

© 한상훈 NestJS 백엔드 마스터

결제사를 부르는 PaymentGateway

payment-gateway.ts

```ts
export class PaymentGateway {
  static created = 0;
  readonly no = ++PaymentGateway.created;

  constructor() {
    console.log(`PaymentGateway #${this.no} 생성`);
  }

  async charge(amount: number) {
    return { approvalId: `pay-${this.no}-${amount}`, amount };
  }

  async refund(approvalId: string) {
    return { refunded: approvalId };
  }
}
```

→ 지금은 가짜: 승인 번호만 생성, 승인 번호에 객체 번호 포함

→ 객체가 생길 때마다 번호 증가와 생성 로그

© 한상훈 NestJS 백엔드 마스터

서비스 안의 new

```ts
// reservations.service.ts
@Injectable()
export class ReservationsService {
  async reserve(seatId: string, amount: number) {
    const gateway = new PaymentGateway();
    const payment = await gateway.charge(amount);
    return { seatId, ...payment };
  }
}
```

```ts
// refunds.service.ts
@Injectable()
export class RefundsService {
  private readonly gateway = new PaymentGateway();

  refund(approvalId: string) {
    return this.gateway.refund(approvalId);
  }
}
```

→ 예매: 메서드 안에서 new / 환불: 필드에서 new

© 한상훈 NestJS 백엔드 마스터

요청 세 번, 결제사 객체 세 개

○ ○ ○ 부팅 로그와 예매 요청

```
LOG [NestFactory] Starting Nest application...
ConcertsService 생성
PaymentGateway #1 생성
SeatsService 생성
...
LOG [NestApplication] Nest application successfully started

$ (POST /reservations 세 번)
{"seatId":"A-3-11","approvalId":"pay-2-110000","amount":110000}
{"seatId":"A-3-12","approvalId":"pay-3-110000","amount":110000}
{"seatId":"A-3-13","approvalId":"pay-4-110000","amount":110000}

PaymentGateway #2 생성
PaymentGateway #3 생성
PaymentGateway #4 생성
```

→ #1: 부팅 때 RefundsService 필드

→ #2~#4: 예매 요청마다 하나씩

© 한상훈 NestJS 백엔드 마스터

요청마다 새 커넥션 풀

→ 실제 결제사·DB 클라이언트: 접속을 열어 두고 재사용하는 풀 보유

→ 요청마다 new: 요청마다 새 풀, 재사용할 접속 없음

→ 매번 새 접속과 TLS, 접속 수 증가

→ Prisma 공식 문서: PrismaClient 인스턴스마다 커넥션 풀 생성, 서버 전체에 하나 권장

데이터 블록: Prisma 공식 문서 Connection management, 출처 문장은 부록 시트(2026-10-26 기준), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

생성자에 인자 두 개

payment-gateway.ts

```
export class PaymentGateway {
  static created = 0;
  readonly no = ++PaymentGateway.created;

  constructor(
    private readonly secretKey: string,
    private readonly timeoutMs: number,
  ) {
    console.log(`PaymentGateway #${this.no} 생성`);
  }
```

→ 실제 결제사: 비밀 키 필수, 응답 대기 시간 지정 필요

© 한상훈 NestJS 백엔드 마스터

new 하던 곳마다 에러

터미널

```
$ npx tsc --noEmit -p apps/ticketing/tsconfig.app.json
apps/ticketing/src/reservations/refunds.service.ts(6,30): error TS2554: Expected 2 arguments, but got 0.
apps/ticketing/src/reservations/reservations.service.ts(7,21): error TS2554: Expected 2 arguments, but got 0.
```

→ 클래스 하나 수정 → 쓰는 파일 두 개 수정

→ 비밀 키를 어디서 읽을지도 파일마다 결정

만드는 법을 아는 곳이 여러 군데

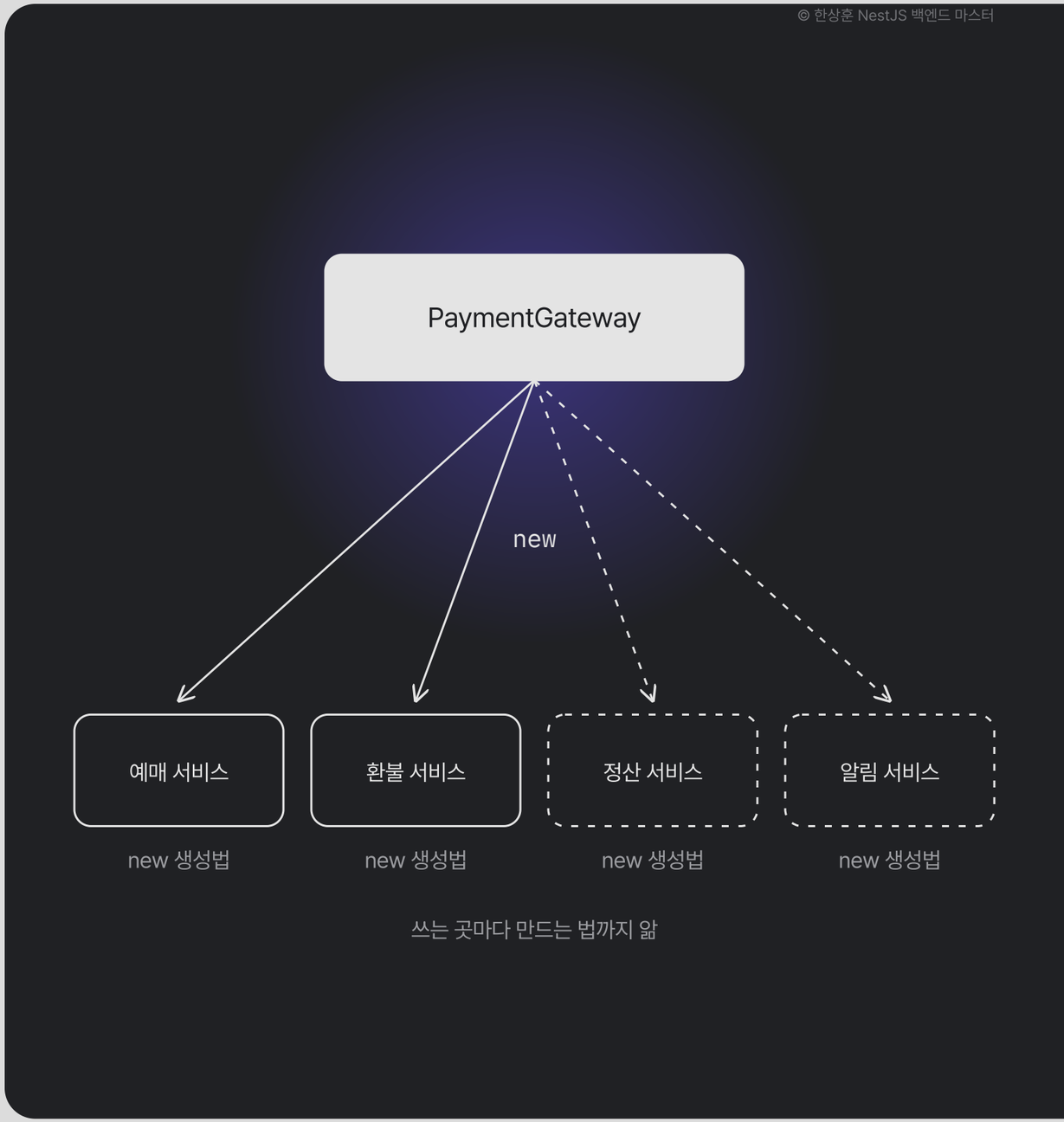
→ 예매 서비스: 결제사 객체 생성법을 앎

→ 환불 서비스: 결제사 객체 생성법을 앎

→ 정산, 알림이 결제사를 쓰기 시작하면: 넷

→ 납품처가 바뀌면 셰프 전원에게 설명

© 한상훈 NestJS 백엔드 마스터



© 한상훈 NestJS 백엔드 마스터

@Injectable과 PaymentsModule

```ts
// payments/payment-gateway.ts
@Injectable()
export class PaymentGateway { ... }

// payments/payments.module.ts
@Module({
  providers: [PaymentGateway],
  exports: [PaymentGateway],
})
export class PaymentsModule {}
```

```ts
// reservations/reservations.module.ts
@Module({
  imports: [PaymentsModule],
  controllers: [ReservationsController],
  providers: [ReservationsService, RefundsService],
})
export class ReservationsModule {}
```

© 한상훈 NestJS 백엔드 마스터

생성자로 받는 두 서비스

```
@Injectable()
export class ReservationsService {
  constructor(private readonly gateway: PaymentGateway) {
    console.log('ReservationsService 생성');
  }

  async reserve(seatId: string, amount: number) {
    const payment = await this.gateway.charge(amount);
    return { seatId, ...payment };
  }
}

@Injectable()
export class RefundsService {
  constructor(private readonly gateway: PaymentGateway) {
    console.log('RefundsService 생성');
  }
```

→ new가 사라진 자리: 생성자 인자 하나

© 한상훈 NestJS 백엔드 마스터

객체 하나, 승인 번호 pay-1

부팅 로그와 예매 요청 세 번

```
LOG [NestFactory] Starting Nest application...
ConcertsService 생성
PaymentGateway #1 생성
ReservationsService 생성
RefundsService 생성
LOG [InstanceLoader] PaymentsModule dependencies initialized
SeatsService 생성
...
{"seatId":"A-3-11","approvalId":"pay-1-110000","amount":110000}
{"seatId":"A-3-12","approvalId":"pay-1-110000","amount":110000}
{"seatId":"A-3-13","approvalId":"pay-1-110000","amount":110000}
```

→ 요청 중 생성 로그 없음

© 한상훈 NestJS 백엔드 마스터

의존성과 의존성 주입

의존성 (dependency)

클래스가 일을 하려면 있어야 하는 다른 객체

ReservationsService의 의존성: `PaymentGateway`

의존성 주입 (dependency injection)

그 객체를 클래스 밖에서 만들어 넣어 주는 방식

Nest의 기본: 생성자 주입, 생성자 인자로 선언

© 한상훈 NestJS 백엔드 마스터

누가 new를 부르나

구분	직접 new	주입
new를 부르는 쪽	내 서비스 코드	Nest
만드는 시점	코드가 실행될 때마다	부팅 때 한 번
몇 개	부른 횟수만큼	서버에 하나(기본)

© 한상훈 NestJS 백엔드 마스터

부팅 로그의 생성 순서

부팅 로그

```
LOG [NestFactory] Starting Nest application...
ConcertsService 생성
PaymentGateway #1 생성          ← 필요한 쪽보다 먼저
ReservationsService 생성
RefundsService 생성
LOG [InstanceLoader] PaymentsModule dependencies initialized
SeatsService 생성
LOG [InstanceLoader] AppModule dependencies initialized
LOG [InstanceLoader] ReservationsModule dependencies initialized
...
LOG [NestApplication] Nest application successfully started
```

→ 의존성 먼저, 그다음 의존성을 받는 서비스

→ `listen` 전에 전부 생성

© 한상훈 NestJS 백엔드 마스터

제어의 역전과 IoC 컨테이너

→ 제어의 역전(inversion of control): 객체 생성과 호출 시점의 결정권이 내 코드에서 프레임워크로

→ IoC 컨테이너: 프로바이더를 만들고 보관하다가 생성자에 넣어 주는 Nest 내부

→ 필요한 객체를 못 찾으면: 첫 요청 전, 부팅에서 중단

→ 컨트롤러 메서드 호출도 같은 구조: 내가 부르지 않고 Nest가 호출

© 한상훈 NestJS 백엔드 마스터

실행 중에 사라지는 타입

→ TypeScript 타입: 빌드 후 JavaScript에서 삭제

→ `constructor(private readonly gateway: PaymentGateway)` → `constructor(gateway)`

→ 그런데 Nest는 PaymentGateway를 앎: 빌드가 따로 남긴 정보

→ tsconfig.json: `"emitDecoratorMetadata": true`, `"experimentalDecorators": true`

© 한상훈 NestJS 백엔드 마스터

dist에 남은 design:paramtypes

dist/apps/ticketing/main.js

```
ReservationsService = _ts_decorate([
    (0,_nestjs_common__rspack_import_0.Injectable)(),
    _ts_metadata("design:type", Function),
    _ts_metadata("design:paramtypes", [
        typeof _payments_payment_gateway_js__rspack_import_1.PaymentGateway === "undefined" ? Object : _payments_payment_gateway_js__rspack_import_1.PaymentGateway
    ])
], ReservationsService);
```

→ `design:paramtypes`: 생성자 인자 타입 목록, 실행 코드에 남은 클래스

→ 데코레이터가 붙은 클래스만 기록

→ Nest: 목록의 PaymentGateway를 열쇠로 프로바이더 검색

© 한상훈 NestJS 백엔드 마스터

import type 한 단어

reservations.service.ts

```ts
import { Injectable } from '@nestjs/common';
import type { PaymentGateway } from '../payments/payment-gateway.js';

@Injectable()
export class ReservationsService {
  constructor(private readonly gateway: PaymentGateway) {
```

→ `import type`: 타입으로만 쓰는 import, 빌드 후 삭제

→ 에디터·린트 자동 수정으로 바뀌는 경우

© 한상훈 NestJS 백엔드 마스터

tsc는 통과, 부팅은 실패

○ ○ ○ 터미널

```
$ npx tsc --noEmit -p apps/ticketing/tsconfig.app.json
tsc exit 0

$ node dist/apps/ticketing/main.js
ERROR [ExceptionHandler] UnknownDependenciesException [Error]: Nest can't resolve dependencies of the
ReservationsService (?). Please make sure that the argument at index [0] is available in the current module.

Potential solutions:
- The dependency at index [0] appears to be undefined at runtime
- This commonly occurs when using 'import type' instead of 'import' for injectable classes
```

© 한상훈 NestJS 백엔드 마스터

Object가 남은 메타데이터

dist/apps/ticketing/main.js

```
ReservationsService = _ts_decorate([
    (0,_nestjs_common__rspack_import_0.Injectable)(),
    _ts_metadata("design:type", Function),
    _ts_metadata("design:paramtypes", [
        typeof PaymentGateway === "undefined" ? Object : PaymentGateway
    ])
], ReservationsService);
```

→ 실행 코드에 PaymentGateway 없음 → `Object`

→ Object를 열쇠로 찾는 프로바이더 없음 → 부팅 에러

© 한상훈 NestJS 백엔드 마스터

Nest 없이 서비스 테스트

reservations.service.spec.ts

```ts
import { ReservationsService } from './reservations.service.js';
import { PaymentGateway } from '../payments/payment-gateway.js';

it('결제사를 실제로 부르지 않고 예매를 시험한다', async () => {
  const fakeGateway = {
    charge: async (amount: number) => ({ approvalId: 'fake-1', amount }),
  } as PaymentGateway;

  const service = new ReservationsService(fakeGateway);

  expect(await service.reserve('A-3-15', 110000)).toEqual({
    seatId: 'A-3-15',
    approvalId: 'fake-1',
    amount: 110000,
  });
});
```

© 한상훈 NestJS 백엔드 마스터

vitest 결과

터미널

```
$ npx vitest run apps/ticketing/src/reservations
 RUN  v4.1.12

 Test Files  1 passed (1)
      Tests  1 passed (1)
   Duration  511ms
```

→ 결제사 호출 없음, Nest 부팅 없음

데이터 블록: vitest 4.1.12, 511ms, 2026-10-26 강사 PC 실측(Node v24.11.0), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

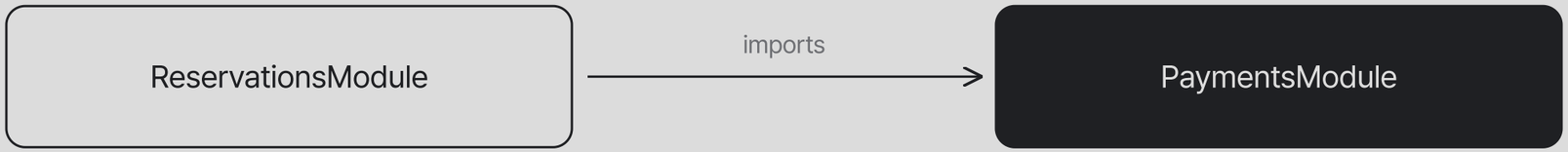
new와 주입 비교

구분	서비스 안에서 new	생성자로 받기
결제사 객체 수	요청마다 하나	서버에 하나
생성자 변경 시 고칠 곳	쓰는 파일 전부	만드는 곳 한 군데
테스트	진짜 결제사 호출	가짜를 넣어서
빠진 의존성 발견	그 줄이 실행될 때	부팅 때

© 한상훈 NestJS 백엔드 마스터

받기만 하는 쪽

→ 모듈 그래프: ReservationsModule → PaymentsModule



"객체를 쓰는 쪽은 받기만 하고, 만드는 일은 Nest가 맡습니다."